Line Guardian: what was required and what we did

Exercise 5 | Required Stages A and B | 11 November 2026

1. What did the assignment require?

Investigate whether the current predictive-maintenance system is good enough. Today, plant gateways write sensor readings into PostgreSQL and one detector polls the table. The model needs every reading of each machine in sequence order, and scoring costs 3 ms per reading.

Stage A: run a small plant (20 machines, 200 readings/s) for about 90 seconds, allow the detector to drain, and compare its count with the database. Then run the full plant (60 machines, 600 readings/s) for about 60 seconds and observe delay, backlog and missing readings while it runs.

Stage B: compare improved PostgreSQL polling with a Kafka-based design against R1-R4. Write a short decision record with the choice, reasons and detector-instance estimates. Explain when simple PostgreSQL polling would be sufficient.

Optional Stage C: implement the Kafka sink and detector, then validate the result. We skipped this as requested.

2. What did we actually do?

Prepared the Python virtual environment and installed the exercise dependencies. Started PostgreSQL, Kafka and Kafka UI, and stopped the earlier exercise container that occupied port 5432. Ran both Stage A experiments using the supplied code, saved the output in Exc_5/results/, and wrote Exc_5/decision-record.md. No Python source files were edited.

Measured result	Small plant	Full plant
Run duration / input rate	90 s / 200 readings/s	60 s / 600 readings/s
Rows sent and stored	18,000	36,000
Detector readings at stop	15,405	16,801
Model missing counter	2,585	50
Last periodic backlog	0 after draining	18,490
Last periodic delay	0.0 s when idle	30.9 s

The full detector stopped shortly after the plant. Its last periodic backlog and delay are snapshots before shutdown. Idle delay is printed as zero by the supplied code; it is not a measured p95 latency.

3. Which questions do the observations answer?

Did the small detector read everything? No. All 18,000 rows reached PostgreSQL, but only 15,405 were read: 2,595 stored readings were skipped despite a final backlog of zero. The model counted 2,585 missing; its counter cannot identify gaps before the first reading it sees for a machine or after the last.

Why are rows skipped? Sequence IDs are allocated before commit. Gateway 1 can insert IDs 100-114 and commit late, while gateway 2 commits IDs 115-129 first. The detector sees the latter and remembers 129. Its next query asks for IDs above 129, permanently excluding 100-114. The rows exist in PostgreSQL; the polling cursor is unsafe.

What changes with the full plant? Backlog and delay grow because a single detector cannot score 600 readings/s. In the last five reports, backlog grew from 9,690 to 18,490 and delay from 16.3 to 30.9 seconds. The missing counter stayed at 50 over those reports: the backlog gives late transactions more time to commit, partly masking the cursor problem. It does not solve it.

4. What decision did we make?

Requirement	Assessment and proposed response
R1: 600/s, later 100,000/s	Use Kafka partitions and multiple detector instances. Database, broker, network and storage capacity still require benchmarking.
R2: complete and ordered	Key readings by machine_id; process each partition sequentially. Use reliable delivery and recover model state consistently with offsets.
R3: p95 delay at most 5 s	Provision inference headroom and monitor event age. The baseline falls behind; the proposed Kafka design has not been implemented or validated.
R4: independent consumers	Use separate groups for detector, archive and dashboard. Extra readers consume Kafka rather than adding raw-reading queries to PostgreSQL.

Choice: Kafka for raw readings; PostgreSQL stays the system of record for the machine registry. Adding PostgreSQL pollers alone would not fix the ID-cursor bug, and additional polling consumers would add database load.

Capacity calculation: one sequential instance has an ideal ceiling of 1 / 0.003 = about 333 readings/s. The inference-only minimum is 2 instances at 600/s and 300 at 100,000/s. At 70% target utilisation, budget about 3 and 429, respectively, with sufficient partitions. These estimates exclude overhead, imbalance and recovery costs.

When is PostgreSQL enough? A small office sending one temperature reading per minute does not need this scale. The one-worker inference-only tipping point is roughly 333 readings/s, or 233/s at 70% utilisation. Correctness and independent-reader requirements may force a redesign earlier.

Completion status: required experiments and written analysis completed locally. Optional Stage C skipped. No Kafka detector implemented; no 5-second p95 proof. GitHub push and Moodle submission have not been performed.